



ESCUELA POLITÉCNICA NACIONAL
FACULTAD DE INGENIERÍA EN SISTEMAS
INGENIERIA EN SISTEMAS

AUTOMATIZACIÓN DE CAPTURA Y DOCUMENTACIÓN
DE SESIONES LINUX MEDIANTE MULTIPROCESAMIENTOS

SISTEMAS OPERATIVOS

Diego Pacheco

Alexander Ponce

Danny Ponce

Jorge Puetate

GRUPO 4

31 de Mayo de 2026

OBJETIVOS:



Objetivo General:

Desarrollar una herramienta automatizada basada en el lenguaje C que capture, procese de forma concurrente y publique registros de sesiones nativas de Linux en un entorno web dinámico.

Objetivos Específicos:

- Implementar un modelo de concurrencia basado en tres hilos paralelos para segmentar las etapas de limpieza, normalización y transformación de datos.
- Garantizar la exclusión mutua y un flujo de control determinista mediante el uso de semáforos, evitando condiciones de carrera y bloqueos mutuos.
- Automatizar el despliegue del reporte utilizando procesos hijos para invocar utilidades externas del sistema operativo.

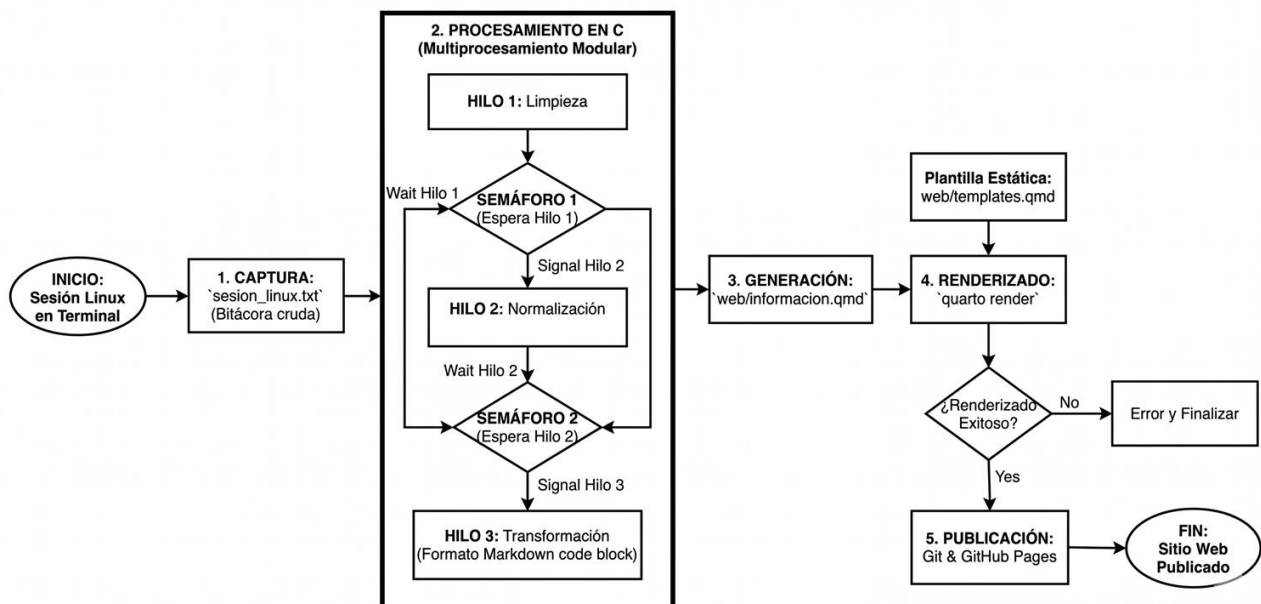
INTRODUCCIÓN

El registro de sesiones de terminal en Linux es crucial para la auditoría de servidores, pero la salida en crudo contiene secuencias de escape que dificultan su lectura, para darle una solución se elaboró una herramienta desarrollada en C que automatiza el procesamiento de estas bitácoras, el sistema captura una sesión nativa, limpia las impurezas del texto y compila dinámicamente los datos en una página web interactiva utilizando Quarto, todo esto gracias a la orquestación de procesos y la sincronización de hilos

DESARROLLO

El flujo general del sistema fue diseñado de manera modular, iniciando con la captura de la sesión Linux mediante el uso de herramientas nativas para generar una bitácora en crudo, posteriormente, esta bitácora ingresa al procesamiento, donde un ejecutable orquestador delega la lectura y limpieza de caracteres a un motor de concurrencia basado en hilos. Una vez que el texto es encapsulado, se da paso a la generación de archivos Markdown/Quarto, donde el orquestador invoca la compilación final que unifica los estilos visuales con los datos. Finalmente, se procede con la publicación web del archivo HTML resultante utilizando Quarto PUB y GitHub Pages.

DIAGRAMA DE FLUJO SISTEMA SHELLDOC WEB





El desarrollo de la solución se fundamenta en cuatro pilares técnicos del diseño de sistemas operativos:

- **Parametrización del código:** El programa orquestador (`main.c`) recibe los nombres de los archivos dinámicamente desde la terminal utilizando los argumentos `argc` y `argv[]`, permitiendo procesar diferentes bitácoras sin necesidad de recompilar.
- **Archivos de entrada/salida (E/S):** El acceso a disco se maneja mediante la librería estándar. El programa utiliza `fopen()` en modo lectura para ingerir la bitácora original, y en modo escritura para inyectar el texto limpio junto con las etiquetas Markdown (`bash`) en el archivo de salida `informacion.qmd`, cerrando de forma segura los punteros con `fclose()`.
- **Uso de hilos (Threads):** El procesamiento pesado del texto recae en la librería `pthread`. Se diseñaron tres hilos paralelos (Limpieza, Normalización y Transformación) que operan sobre arreglos de memoria compartida estáticos de gran capacidad. Para evitar condiciones de carrera (*race conditions*), se utilizan dos semáforos POSIX (`sem_wait` y `sem_post`) que obligan a los hilos a ejecutarse en un orden estrictamente secuencial.
- **Uso de `fork()`:** El control del flujo general está manejado por procesos. El proceso padre (orquestador) invoca `fork()` para crear un proceso hijo aislado que ejecute el motor de hilos mediante `execvp()`. El padre se bloquea de forma segura usando `waitpid()` hasta que el hijo termina de escribir el archivo. Posteriormente, se lanza un segundo `fork()` para delegar la ejecución externa del comando `quarto render`, protegiendo en todo momento la integridad del hilo principal.

A continuación, se demuestra la funcionalidad del sistema operando en el entorno del servidor Ubuntu.

A. Ejecución de comandos parametrizados: Captura del orquestador invocando los archivos de entrada y la sincronización exitosa de los hilos.

```
dannuki@dany-server:~/Proyecto_ShellDoc_Web$ ./Iniciador sesion_linux.txt web/templates.qmd
shellDoc Web - Orquestador
] Sesión : sesion_linux.txt
Render : web/templates.qmd

[FASE 1] Lanzando procesador de hilos...
Hilo 1 terminado
Hilo 2 terminado
Archivo informacion.qmd generado
Proceso completado
[OK] Hilos completados. informacion.qmd generado.
```

B. Archivos generados: Evidencia de la correcta escritura E/S del archivo de salida `informacion.qmd` con el texto limpio y el formato Markdown.



```
1 title: "ShellDoc Web"
2 format: html
3 ---
4
5 # Sesion Linux
6
7 ``bash
8
9 Script started on 2026-05-26 02:54:11+00:00 [TERM="linux" TTY="/dev/tty2" COLUMNS="160" LINES="50"]
10 [?2004hpdiego2026a@diego:~$ uname -a
11 [?2004lLinux diego 7.0.0-14-generic #14-Ubuntu SMP PREEMPT_DYNAMIC Mon Apr 13 11:09:53 UTC 2026 x86_64 GNU/Linux:
12 [?2004hpdiego2026a@diego:~$
13 [?2004l[?2004hpdiego2026a@diego:~$ df -h
14 [?2004lFilesystem                                Size  Used Avail Use% Mounted on
15 tmpfs                                           872M  1,1M  871M   1% /run
16 /dev/mapper/ubuntu--vg-ubuntu--lv             50G   9,0G   38G  20% /
17 tmpfs                                           2,2G   0  2,2G   0% /dev/shm
18 none                                           1,0M   0  1,0M   0% /run/credentials/systemd-journald.service
19 none                                           1,0M   0  1,0M   0% /run/credentials/systemd-resolved.service
20 tmpfs                                           2,2G   0  2,2G   0% /tmp
21 /dev/sda2                                       2,0G  123M   1,7G   7% /boot
22 none                                           1,0M   0  1,0M   0% /run/credentials/systemd-networkd.service
23 none                                           1,0M   0  1,0M   0% /run/credentials/getty@tty1.service
24 tmpfs                                           436M   8,0K  436M   1% /run/user/1000
25 none                                           1,0M   0  1,0M   0% /run/credentials/getty@tty2.service
26 none                                           1,0M   0  1,0M   0% /run/credentials/getty@tty3.service
27 none                                           1,0M   0  1,0M   0% /run/credentials/getty@tty4.service
28 none                                           1,0M   0  1,0M   0% /run/credentials/getty@tty5.service
29 none                                           1,0M   0  1,0M   0% /run/credentials/getty@tty6.service
30 [?2004hpdiego2026a@diego:~$ top -b
31 [?2004ltop - 02:55:39 up 1:27, 2 users, load average: 0,10, 0,11, 0,05
32 Tasks: 139 total, 1 running, 138 sleeping, 0 stopped, 0 zombie
33 %Cpu(s): 2,6 us, 0,0 sy, 0,0 ni, 94,7 id, 2,6 wa, 0,0 hi, 0,0 si, 0,0 st
```

C. Visualización en Quarto: Resultado final renderizado, demostrando la integración de la bitácora con los estilos del sitio web en formato HTML.



documentación se público en github pages, se puede acceder mediante este enlace:
https://dannuki.github.io/Proyecto_ShellDoc_Web/

CONCLUSIÓN

El desarrollo de **ShellDoc Web** demostró la viabilidad de emplear el lenguaje C para el diseño de soluciones robustas de automatización de bajo nivel. Mediante un modelo estricto de exclusión mutua basado en semáforos lógicos, se garantizó la sincronización determinista de tres hilos paralelos sobre memoria compartida, erradicando por completo condiciones de carrera y bloqueos mutuos. Asimismo, la arquitectura modular implementada mediante multiprocesamiento (fork, exec l p y w a i t p i d) probó ser altamente eficiente para el aislamiento de fallos, protegiendo la integridad del orquestador principal al delegar las



tareas críticas de procesamiento de texto y renderizado externo a procesos hijos. El sistema final logra puentear con éxito la captura de bitácoras nativas de Linux con herramientas modernas de alto nivel como Quarto y plataformas de despliegue continuo en la nube (GitHub Pages / Quarto Pub), eliminando la intervención manual y optimizando los procesos de auditoría de servidores.